Lab 7: Fuzzing for Input Validation Bugs

Project: **L6_L7_17_electricity_bill** (Lab 7 / `electricity_bill.py`)

Name: Sarva Sandip SRN: PES1UG24AM255 Section: E

Tools: Python 3.12.3, Hypothesis 6.170.15, pytest 9.2.0

Step 1: Install dependencies

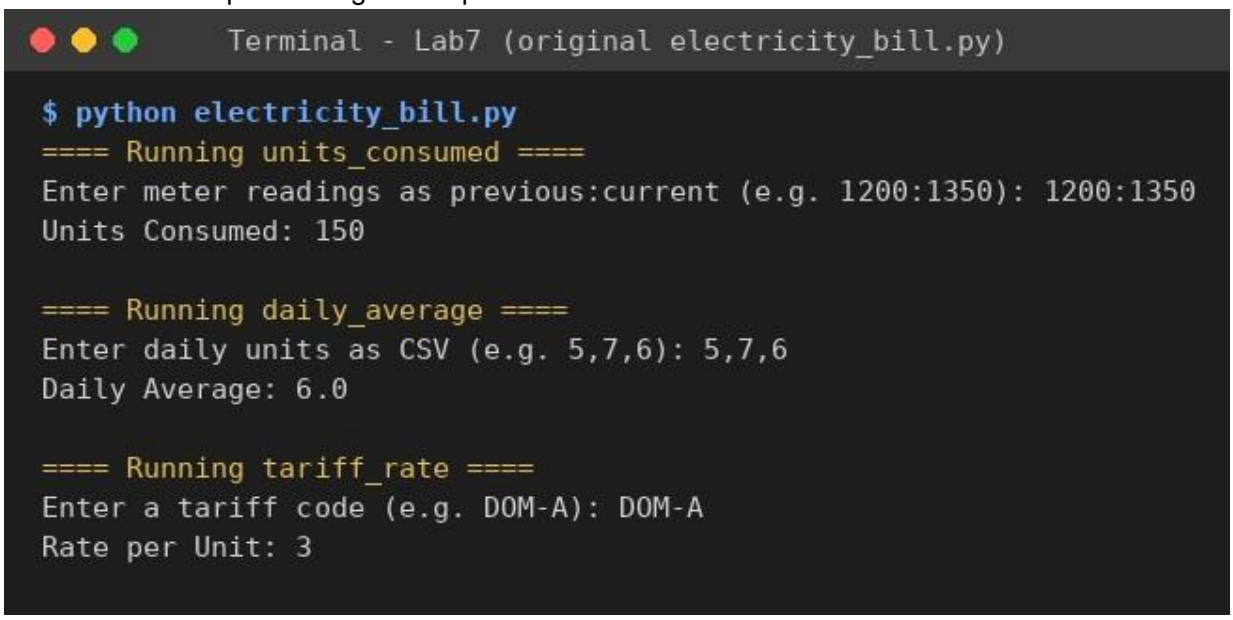
```
pip install hypothesis
pip install pytest
```

Both packages installed successfully (Hypothesis 6.170.15, pytest 9.2.0).

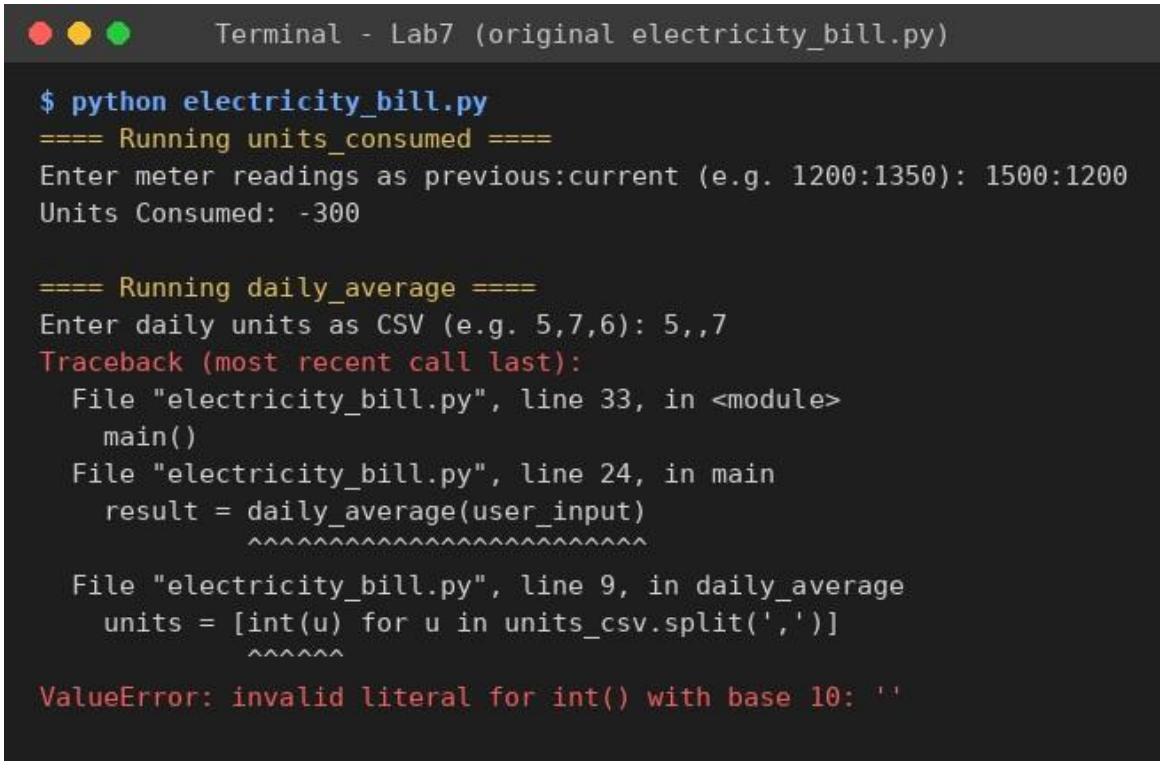
Step 2: Run the buggy application

```
python electricity_bill.py
```

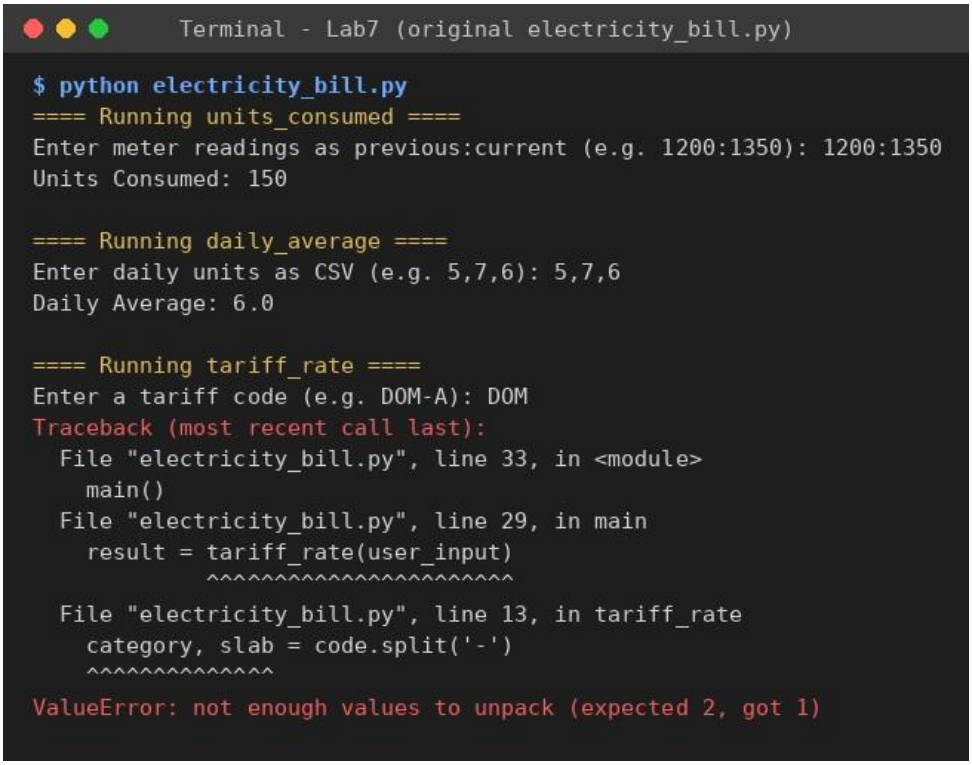
With well-formed input the original script works:



With bad input it either gives a wrong answer silently or crashes with a traceback. Note that **1500:1200** returns **-300** units (a meter running backwards), and **5,,7** raises a ValueError:



A malformed tariff code (**DOM**, no slab) also crashes the program:



Step 3: Complete test_electricity_bill.py with Hypothesis

The three required no-crash fuzz tests were completed using `@given(st.text() | st.none())`. Six extra tests were added to find more edge cases: an any-type fuzz test (ints, floats, bytes, lists, bools), correctness properties for valid input, and checks that invalid input is rejected. The full code is in the submitted *test_electricity_bill.py*.

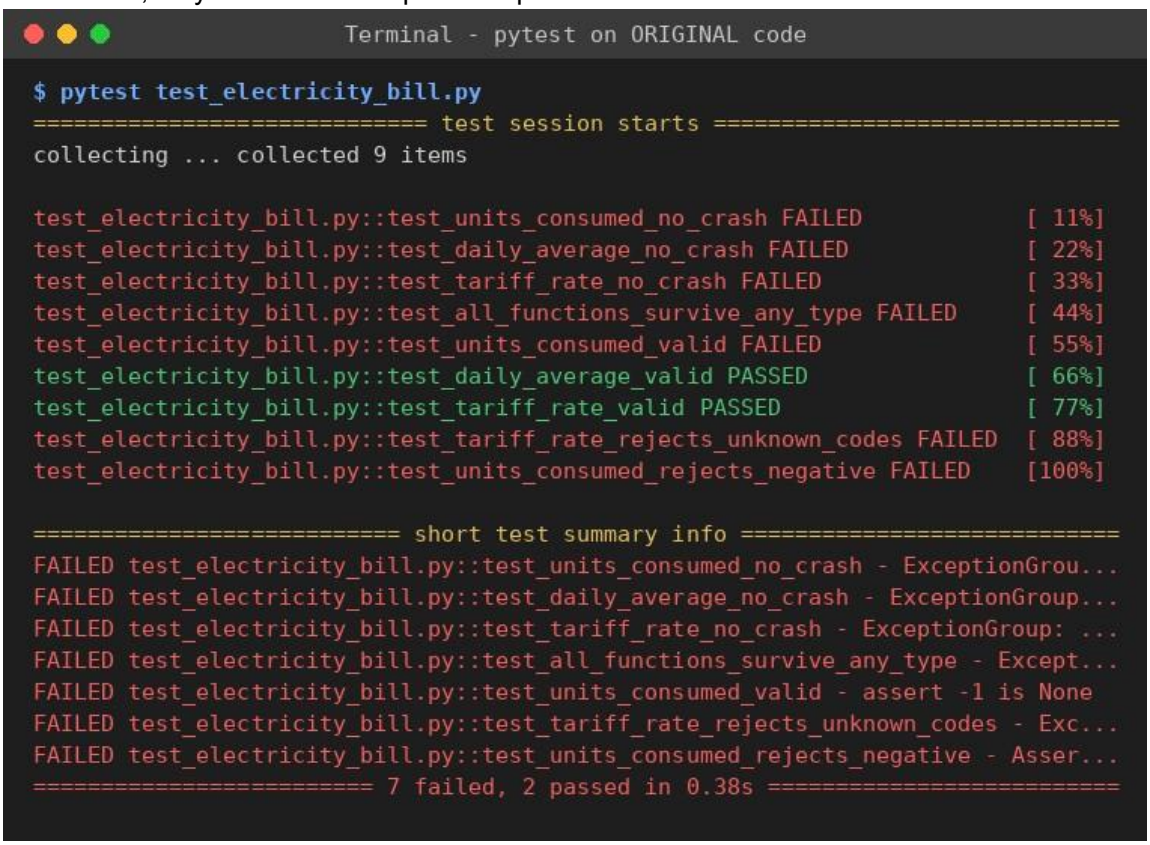
Test	Purpose
test_units_consumed_no_crash	units_consumed must never raise for any text or None
test_daily_average_no_crash	daily_average must never raise for any text or None
test_tariff_rate_no_crash	tariff_rate must never raise for any text or None
test_all_functions_survive_any_type	All 3 functions survive ints, floats, bytes, bools, lists
test_units_consumed_valid	Valid readings give current - previous; backwards meter gives None
test_daily_average_valid	Result equals rounded mean and lies between min and max

test_tariff_rate_valid	Every valid code, in any case, gives RATES[cat] * SLAB[slab]
test_tariff_rate_rejects_unknown_codes	Anything not in the valid-code set returns None
test_units_consumed_rejects_negative	Negative readings return None

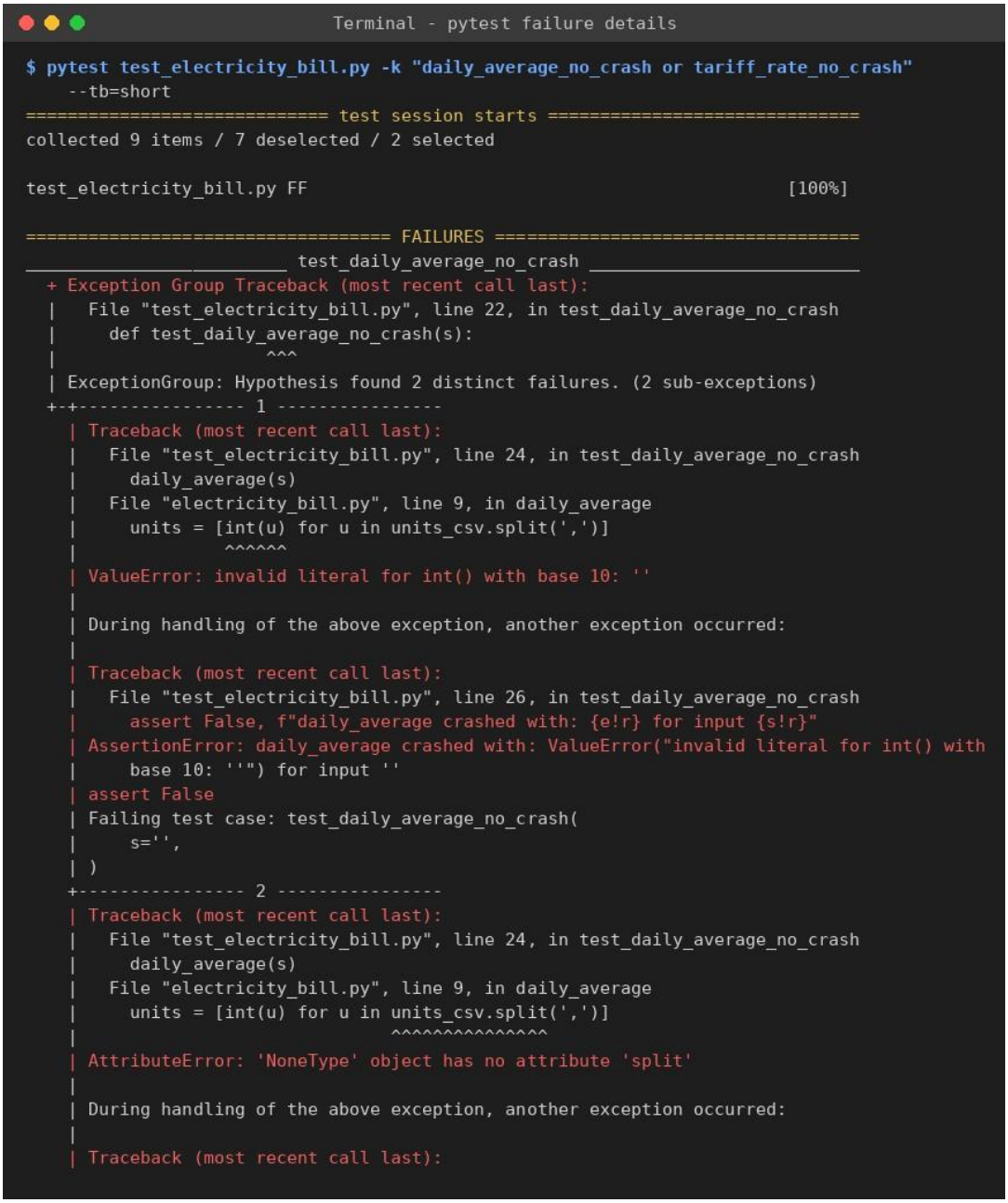
Step 4: Run the tests against the original (buggy) code

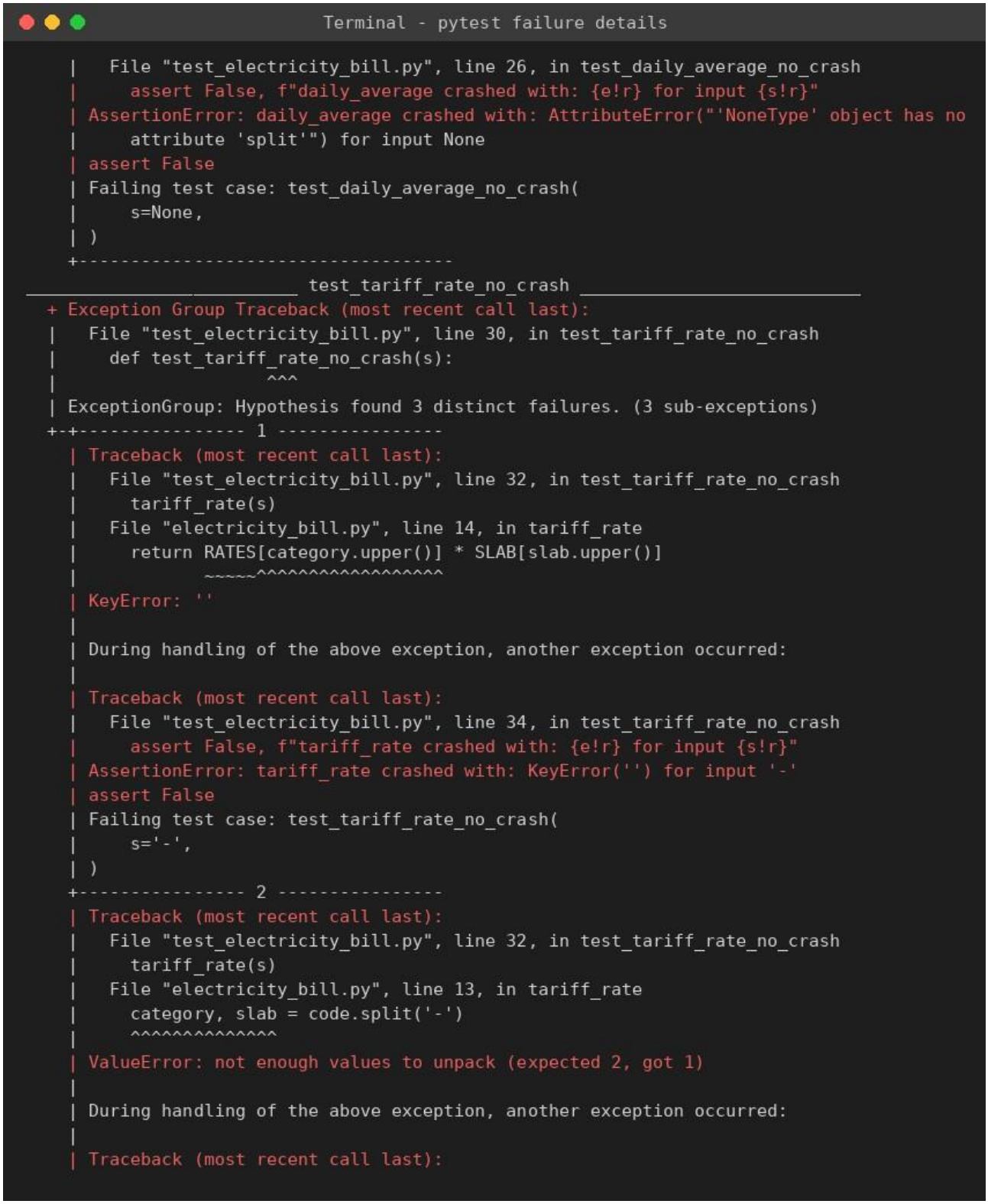
```
pytest test_electricity_bill.py
```

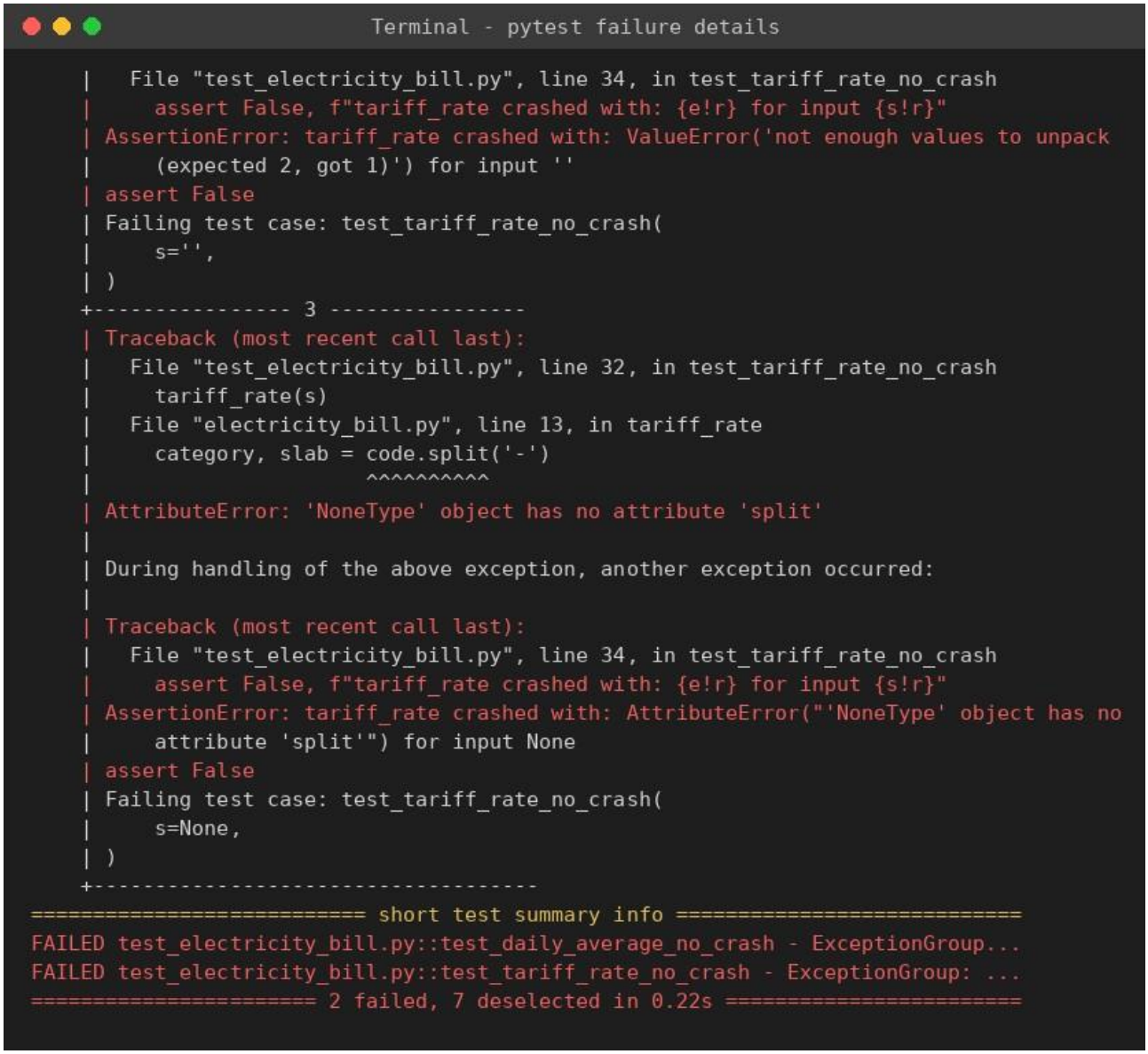
7 of 9 tests fail; only the two valid-input tests pass.



Detailed Hypothesis counterexamples (shrunk to the smallest failing input):


Terminal - pytest failure details
$ pytest test_electricity_bill.py -k "daily_average_no_crash or tariff_rate_no_crash"
--tb=short
============================= test session starts ==============================
collected 9 items / 7 deselected / 2 selected
test_electricity_bill.py FF [100%]
================================== FAILURES ===================================
_____ test_daily_average_no_crash _____
+ Exception Group Traceback (most recent call last):
| File "test_electricity_bill.py", line 22, in test_daily_average_no_crash
| def test_daily_average_no_crash(s):
| ^^^
| ExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exceptions)
+-+---------- 1 ----------
| Traceback (most recent call last):
| File "test_electricity_bill.py", line 24, in test_daily_average_no_crash
| daily_average(s)
| File "electricity_bill.py", line 9, in daily_average
| units = [int(u) for u in units_csv.split(',')]
| ^^^^^^
| ValueError: invalid literal for int() with base 10: ''
|
| During handling of the above exception, another exception occurred:
|
| Traceback (most recent call last):
| File "test_electricity_bill.py", line 26, in test_daily_average_no_crash
| assert False, f"daily_average crashed with: {e!r} for input {s!r}"
| AssertionError: daily_average crashed with: ValueError("invalid literal for int() with
| base 10: ''") for input ''
| assert False
| Failing test case: test_daily_average_no_crash(
| s='',
|)
+---------- 2 ----------
| Traceback (most recent call last):
| File "test_electricity_bill.py", line 24, in test_daily_average_no_crash
| daily_average(s)
| File "electricity_bill.py", line 9, in daily_average
| units = [int(u) for u in units_csv.split(',')]
| ^^^^^^^^^^^^^^^
| AttributeError: 'NoneType' object has no attribute 'split'
|
| During handling of the above exception, another exception occurred:
|
| Traceback (most recent call last):


Terminal - pytest failure details
| File "test_electricity_bill.py", line 26, in test_daily_average_no_crash
| assert False, f"daily_average crashed with: {e!r} for input {s!r}"
| AssertionError: daily_average crashed with: AttributeError("'NoneType' object has no
| attribute 'split'") for input None
| assert False
| Failing test case: test_daily_average_no_crash(
| s=None,
|)
+------------------------------------
_____ test_tariff_rate_no_crash _____
+ Exception Group Traceback (most recent call last):
| File "test_electricity_bill.py", line 30, in test_tariff_rate_no_crash
| def test_tariff_rate_no_crash(s):
| ^^^
| ExceptionGroup: Hypothesis found 3 distinct failures. (3 sub-exceptions)
+-+---------- 1 ----------
| Traceback (most recent call last):
| File "test_electricity_bill.py", line 32, in test_tariff_rate_no_crash
| tariff_rate(s)
| File "electricity_bill.py", line 14, in tariff_rate
| return RATES[category.upper()] * SLAB[slab.upper()]
| ~~~~~^^^^^^^^^^^^^^^^^^^
| KeyError: ''
|
| During handling of the above exception, another exception occurred:
|
| Traceback (most recent call last):
| File "test_electricity_bill.py", line 34, in test_tariff_rate_no_crash
| assert False, f"tariff_rate crashed with: {e!r} for input {s!r}"
| AssertionError: tariff_rate crashed with: KeyError('') for input '-'
| assert False
| Failing test case: test_tariff_rate_no_crash(
| s='-',
|)
+---------- 2 ----------
| Traceback (most recent call last):
| File "test_electricity_bill.py", line 32, in test_tariff_rate_no_crash
| tariff_rate(s)
| File "electricity_bill.py", line 13, in tariff_rate
| category, slab = code.split('-')
| ^^^^^^^^^^^^^^
| ValueError: not enough values to unpack (expected 2, got 1)
|
| During handling of the above exception, another exception occurred:
|
| Traceback (most recent call last):


Terminal - pytest failure details
| File "test_electricity_bill.py", line 34, in test_tariff_rate_no_crash
| assert False, f"tariff_rate crashed with: {e!r} for input {s!r}"
| AssertionError: tariff_rate crashed with: ValueError('not enough values to unpack
| (expected 2, got 1)') for input ''
| assert False
| Failing test case: test_tariff_rate_no_crash(
| s='',
|)
+---------- 3 ----------
| Traceback (most recent call last):
| File "test_electricity_bill.py", line 32, in test_tariff_rate_no_crash
| tariff_rate(s)
| File "electricity_bill.py", line 13, in tariff_rate
| category, slab = code.split('-')
| ^^^^^^^^^^
| AttributeError: 'NoneType' object has no attribute 'split'
|
| During handling of the above exception, another exception occurred:
|
| Traceback (most recent call last):
| File "test_electricity_bill.py", line 34, in test_tariff_rate_no_crash
| assert False, f"tariff_rate crashed with: {e!r} for input {s!r}"
| AssertionError: tariff_rate crashed with: AttributeError("'NoneType' object has no
| attribute 'split'") for input None
| assert False
| Failing test case: test_tariff_rate_no_crash(
| s=None,
|)
+------------------------------------
========================= short test summary info ==========================
FAILED test_electricity_bill.py::test_daily_average_no_crash - ExceptionGroup...
FAILED test_electricity_bill.py::test_tariff_rate_no_crash - ExceptionGroup: ...
====================== 2 failed, 7 deselected in 0.22s ======================

Fix	Reason
isinstance(x, str) check at the top of every function	Handles None, ints, bytes, lists
New helper _parse_units(): strip, ASCII digits only, max 12 digits	Rejects '', '-5', '1.5', '1_0', unicode digits, huge numbers (overflow/DoS)
units_consumed: require exactly 2 parts, reject current < previous	No more unpack crashes or negative usage
daily_average: reject empty list and any empty/invalid element	'' and '1,,3' no longer crash

tariff_rate: exactly 2 parts, dictionary membership check instead of direct indexing	No more KeyError; case and whitespace still tolerated
Invalid input returns None; main() prints 'Invalid input'	Program never crashes on user input

Bugs discovered by fuzzing

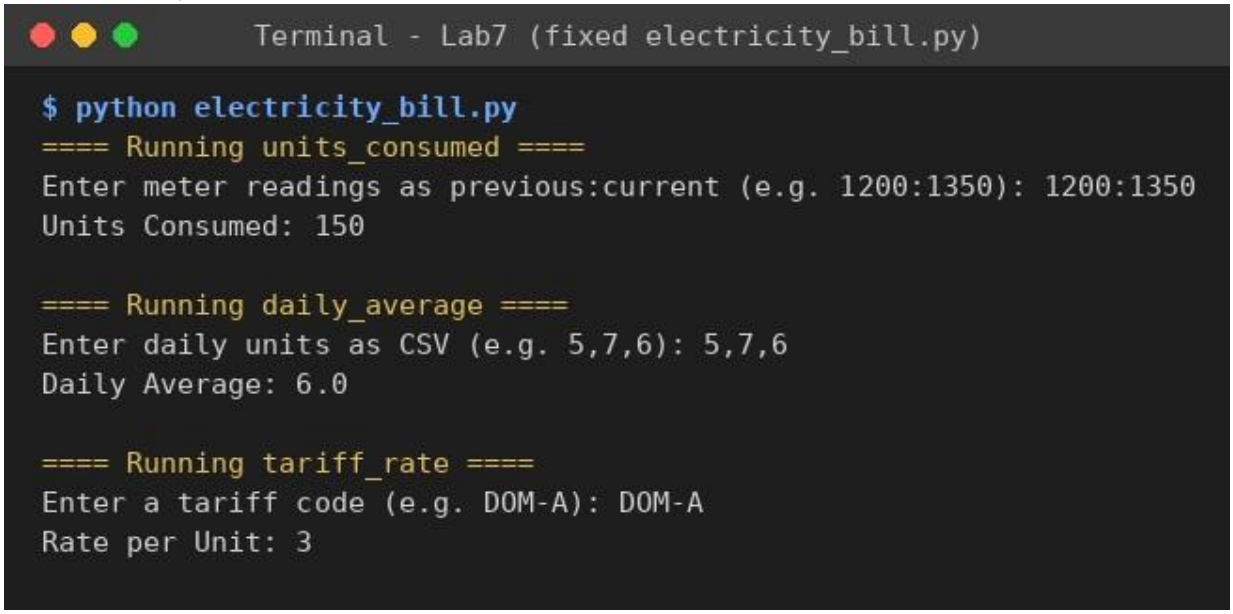
#	Function	Failing input found	Problem
1	units_consumed	None / any non-string	AttributeError: no .split()
2	units_consumed	'' , '5' , '1:2:3'	ValueError: wrong number of ':' parts
3	units_consumed	'a:b' , '1.5:3'	ValueError from int()
4	units_consumed	'0:-1' , '1500:1200'	Negative values accepted; returns a negative usage
5	daily_average	'' , '1,,3'	ValueError from int('')
6	daily_average	None, non-numeric text	AttributeError / ValueError
7	daily_average	'-5,3' , Arabic-Indic digits	Negative or non-ASCII values silently accepted
8	tariff_rate	'' , 'DOM'	ValueError: unpack error
9	tariff_rate	'-' , 'XYZ-A' , 'DOM-C'	KeyError for empty or unknown category/slab
10	tariff_rate	None	AttributeError: no .split()

Step 5: Fix the buggy script

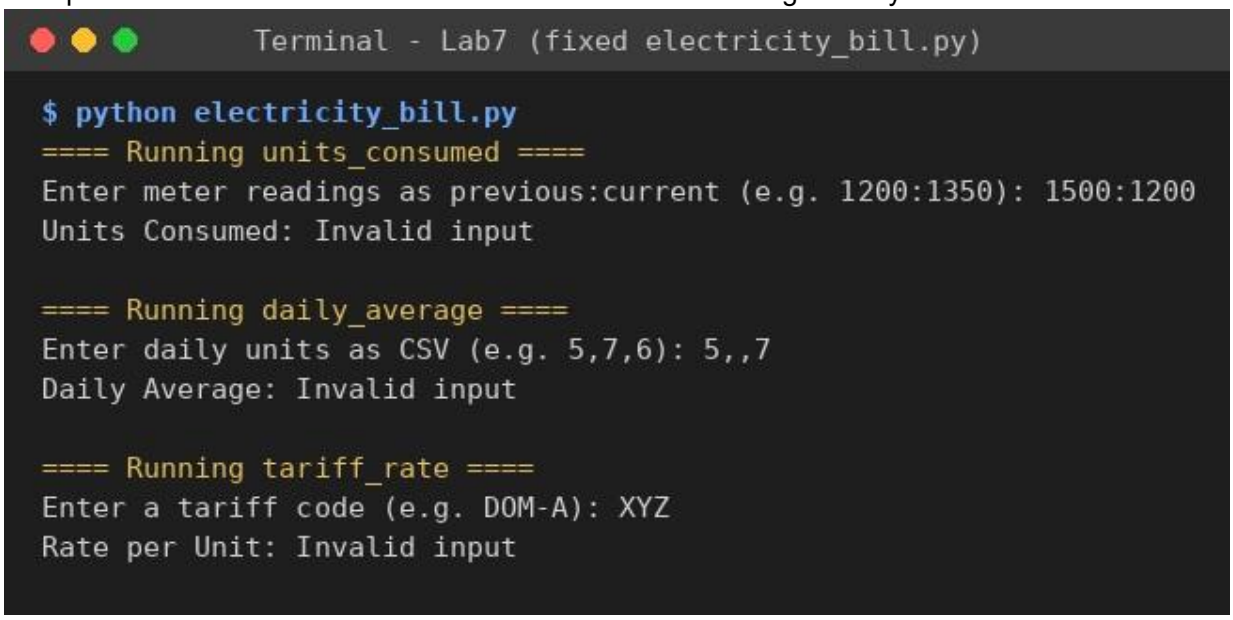
Changes made to *electricity_bill.py* (defensive programming):

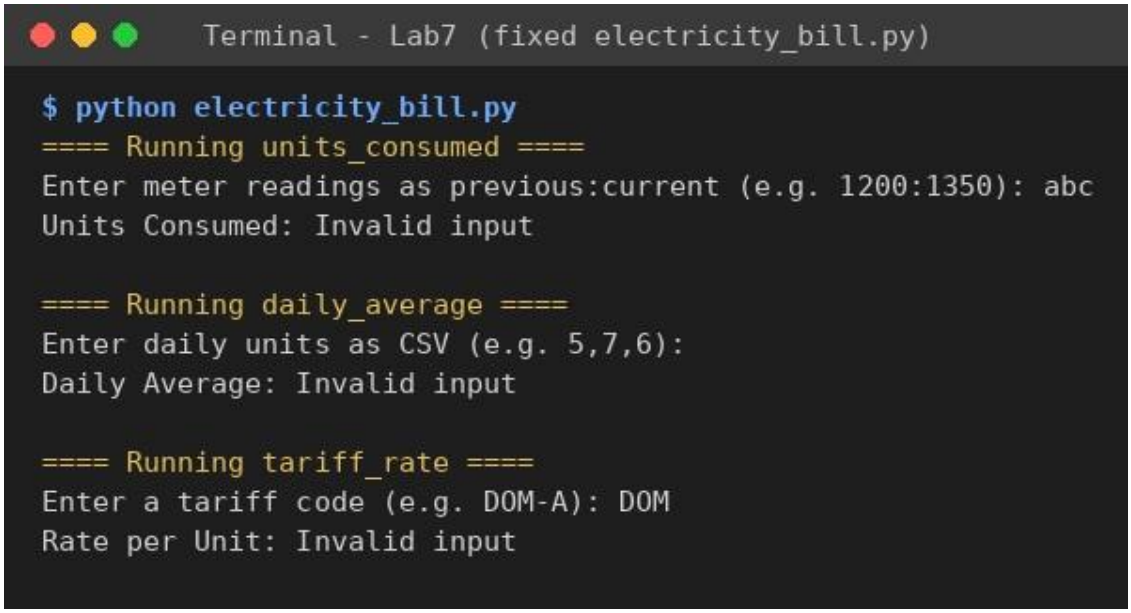
```
python electricity_bill.py
```

Valid input still gives the same results:



The inputs that crashed or misbehaved before are now handled gracefully:

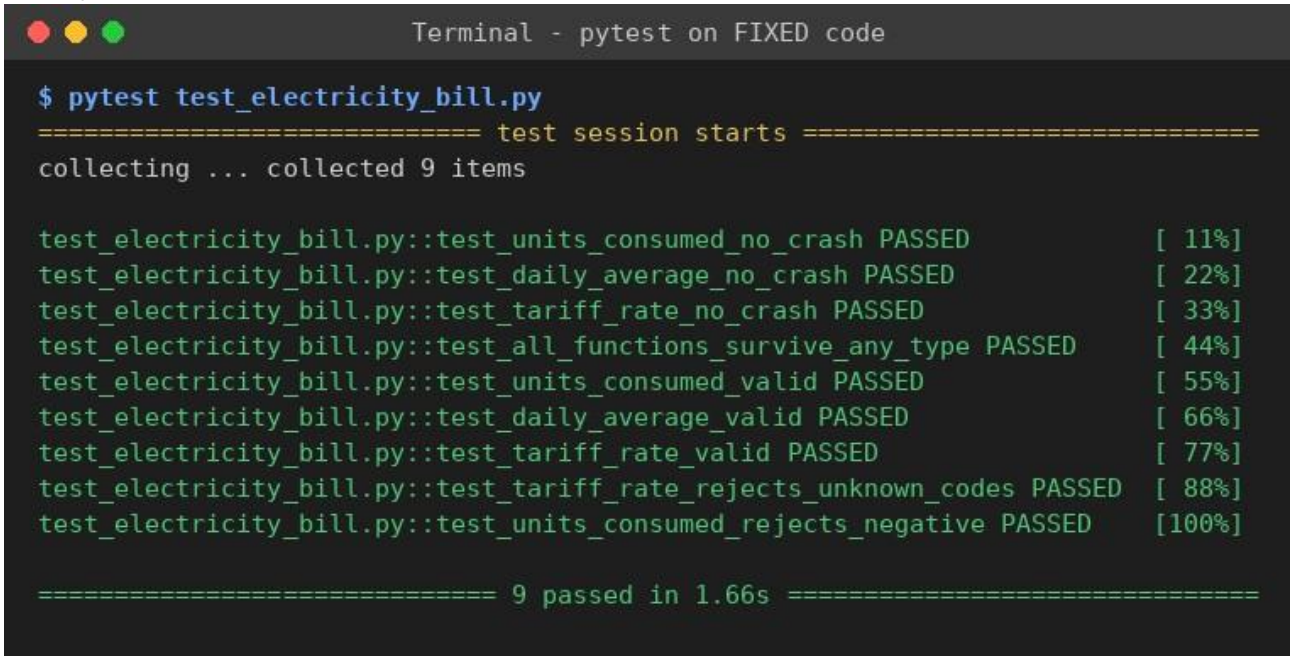



Terminal - Lab7 (fixed electricity_bill.py)
$ python electricity_bill.py
==== Running units_consumed ====
Enter meter readings as previous:current (e.g. 1200:1350): abc
Units Consumed: Invalid input
==== Running daily_average ====
Enter daily units as CSV (e.g. 5,7,6):
Daily Average: Invalid input
==== Running tariff_rate ====
Enter a tariff code (e.g. DOM-A): DOM
Rate per Unit: Invalid input

Step 6: Re-run the tests against the fixed script

```
pytest test_electricity_bill.py
```

All 9 tests pass. (Re-run three more times with different random Hypothesis seeds: 9 passed each time.)



Conclusion

Fuzzing found crashes (ValueError, KeyError, AttributeError) and a logic flaw (negative usage) that a handful of hand-written unit tests would likely have missed. Hardening the functions with strict validation removed all of them, and the property-based tests now guard against regressions.